

dict_config.yaml

```
version: 1
formatters:
  simple:
    format: '%(asctime)s - %(name)s - %(levelname)s -
%(message)s'
handlers:
  console:
    class: logging.StreamHandler
    level: INFO
    formatter: simple
    stream: ext://sys.stdout
loggers:
  sampleLogger:
    level: DEBUG
    handlers: [console]
    propagate: no
root:
  level: DEBUG
  handlers: [console]
```

The output is:

```
2021-11-28 19:25:00,508 -_main_-DEBUG-This is a debug message
2021-11-28 19:25:00,508 -_main_-INFO - This is a info message
2021-11-28 19:25:00,509 -_main_-ERROR-This is a error message
```

 **Note:** When no configuration is provided, the events become an output using a handler of last resort, which is stored in `logging.lastResort`. This acts as `StreamHandler`, which writes messages to `sys.stderr`. Handler's level is set to 'warning' and the behaviour of the logging package in these circumstances is dependent on the Python version. In this article we are using Python 3.2 and above.

Logging from multiple modules

- Calling the same `getLogger("NAME")` multiple times will return a reference to the same object. This can be used across modules to get the same object but requires the same Python interpreter process.
- The application code can define and configure a parent logger in one module, and can only create but not configure a child logger in a separate module. All logger calls to the child will pass up to the parent. The parent and child mapping is done by the name of the logger. Here is an example in the `getLogger` method, where all the configuration is inherited from the parent.

- parent: `getLogger('test_app')`
- child: `getLogger('test_app.sub_app')`

File: main_app.py

```
import sys
import logging
import sub_app
# Create a custom logger with "test_app"
logger = logging.getLogger("test_app")
logger.setLevel(logging.DEBUG)
# Create handlers
# create console handler with debug log level
c_handler = logging.StreamHandler(sys.stdout)
# create file handler with error log level
f_handler = logging.FileHandler('file.log', 'w')
f_handler.setLevel(logging.WARNING)
# Create formatters and add it to handlers
c_format = logging.Formatter('%(name)s - %(levelname)s -
%(message)s')
f_format = logging.Formatter('%(asctime)s - %(name)s -
%(levelname)s - %(message)s')
c_handler.setFormatter(c_format)
f_handler.setFormatter(f_format)
# Add handlers to the logger
logger.addHandler(c_handler)
logger.addHandler(f_handler)
print(c_handler)
print(logger)
logger.debug('This is a debug from main module')
logger.info('This is a info from main module')
logger.warning('This is a warning from main module')
logger.error('This is an error from main module')
logger.info("Before calling the sub_app_function")
sub_app.sub_app_function()
logger.info("After calling the sub_app_function")
```

File: sub_app.py

```
import logging
# create logger
module_logger = logging.getLogger('test_app.sub_app')
def sub_app_function():
    module_logger.error("Debug message from sub_app")
```

The output is:

```
% python multiple_modules_logger.py
<StreamHandler <stdout> (NOTSET)>
<Logger test_app (DEBUG)>
test_app - DEBUG - This is a debug from main module
test_app - INFO - This is a info from main module
test_app - WARNING - This is a warning from main module
test_app - ERROR - This is an error from main module
test_app - INFO - Before calling the sub_app_function
```